

Python – Intermediate Activity Handouts

Activity I1 – Problem Decomposition with Functions

Title: Breaking a Bigger Problem into Small Functions

Context

You now know variables, loops, if/else, and simple functions.

Next: learn to split a problem into clear sub-tasks.

Problem

Given a list of student marks, you must:

- Drop the lowest mark.
- Compute the new average.
- Decide pass/fail using a threshold.

Part A – Think in parts

1. Write the overall problem in one sentence:

2. Identify at least three smaller tasks:

- Task 1: Remove min value from list
- Task 2: Calculate average
- Task 3: compare avg to threshold to see if passed

3. For each task, write a function name and its input and output:

- Function 1 name: drop_lowest (marks)
 - Input(s): list of marks
 - Output: list of marks excluding lowest
- Function 2 name: average (marks)
 - Input(s): list of marks
 - Output: float (average)
- Function 3 name: is_pass (avg, threshold)
 - Input(s): Average, Threshold
 - Output: Boolean (T/F)

Part B – Implement functions

Fill and complete:

python

def drop_lowest(marks):

TODO: remove the smallest mark and return the new list

return new marks

def average(marks):

total = 0

for m in marks:

total = total + m

return total / _____

def is_pass(avg, threshold):

return True or False

if avg >= threshold

return True

else:

return False

Main flow

marks_list = [150, 60, 70, 40]

threshold = 50

new_marks = drop_lowest(marks_list)

avg = average(new_marks)

result = is_pass(avg, threshold)

print("Marks after dropping lowest:", new_marks)

print("Average:", avg)

print("Pass?", result)

Reflection

- How did dividing into functions help you think more clearly?

Dividing the functions make me code more clean-
easy to understand.

Activity I2 – Working with Dictionaries

Title: Simple Student Record System

Problem

You need to manage student data: name, roll number, and marks.
You will use dictionaries for this.

Part A – Design

1. Decide a structure for one student record using a dict:

python

```
student = {  
    "roll": 1,  
    "name": "ABC",  
    "marks": [80, 75, 60]  
}
```

2. What keys will you use?

- Key 1: roll
- Key 2: name
- Key 3: marks

Part B – Code

Complete:

python

```
students = []
```

```
# Add 3 students
```

```
for i in range(3):
```

```
    roll = int(input("Enter roll: "))
```

```
    name = input("Enter name: ")
```

```
    m1 = int(input("Mark 1: "))
```

```
    m2 = int(input("Mark 2: "))
```

```
    m3 = int(input("Mark 3: "))
```

```
    record = {
```

```
        "roll": roll,
```

```
        "name": name,
```

```
        "marks": [m1, m2, m3]
```

```
    }
```

```
    students.append(record)
```

Activity 13 – Reading and Writing Files

Title: Process Data from a Text File

Problem

You are given a text file marks.txt with one mark per line.
You must read it and write a summary to summary.txt.

Part A – Understand

1. Inputs (file/content): marks.txt
2. Outputs (file/content): summary.txt

Part B – Code Skeleton

python

Step 1: Read marks from file

```
marks = []
```

```
with open("marks.txt", "r") as f:
```

```
    for line in f:
```

```
        line = line.strip()
```

```
        if line != "":
```

```
            marks.append(int(line))
```

Step 2: Compute statistics

```
total = sum(marks)
```

```
count = len(marks)
```

```
average = total / count
```

```
max_mark = max(marks)
```

```
min_mark = min(marks)
```

Step 3: Write summary to new file

```
with open("summary.txt", "w") as f:
```

```
    f.write("Number of students: " + str(count) + "\n")
```

```
    f.write("Average mark: " + str(average) + "\n")
```

```
    f.write("Max mark: " + str(max_mark) + "\n")
```

```
    f.write("Min mark: " + str(min_mark) + "\n")
```

Tasks

- Add one more line to summary: count of students with mark ≥ 60 .
- Write that line in the file.

Reflection

- How is processing from file different from input()?

input takes data manually from the user
files are useful for large data & repeated use

Activity 14 – Using Libraries (math / statistics)

Title: Use Standard Library for Cleaner Code

Problem

You need to analyze a list of numbers with mean, median, and standard deviation.

Part A – Manual vs Library

1. In simple words, what do these mean:

- mean: average
- median: middle value

2. Write plain Python code (no extra library) to compute mean.

python

```
numbers = [10, 20, 30, 40, 50]
```

```
# Your mean logic here
```

Part B – Use statistics module

python

```
import statistics as stats
```

```
numbers = [10, 20, 30, 40, 50]
```

```
mean_value = stats.mean(numbers)
```

```
median_value = stats.median(numbers)
```

```
stdev_value = stats.pstdev(numbers)
```

```
print("Mean:", mean_value)
```

```
print("Median:", median_value)
```

```
print("Std Dev:", stdev_value)
```

Task

- Add code to ask the user for numbers (comma-separated) and then compute these three values using the library.

```
import statistics as stats  
num = input("Enter no. separated by comma")  
number = int (map (int, num, split (",",)))  
print (stats, mean (number))  
print (stats, median (number))  
print (stats, S.Dmode (number))
```

q/p

10 ; 20 , 30 , 40

mean : 30

median = 30

std deviation = 14.14

o/p

Activity 15 – Problem Template with Complexity Awareness

Title: Compare Two Approaches (Efficiency)

Problem

You must check if a number is present in a very large list (e.g., 1 million items).

Part A – Two Approaches

Approach 1: Simple loop search.

Approach 2: Use a set.

1. Describe Approach 1 in words:

loop checks each element

2. Describe Approach 2 in words:

set allows fewer lookup.

Part B – Implement Both

python

import time

Create large list

nums = list(range(1000000))

target = 999999

Approach 1: List search

start = time.time()

found = False

for x in nums:

if x == target:

found = True

break

end = time.time()

print("List search found:", found, "Time:", end - start)

Approach 2: Set search

num_set = set(nums)

start = time.time()

found2 = (target in num_set)

end = time.time()

print("Set search found:", found2, "Time:", end - start)

Activity I6 – Input Validation and Error Handling

Title: Robust User Input

Problem

You want a safe function that always returns a valid integer entered by the user.

Part A – Requirements

1. What should happen if user types "abc"?

Show error for "abc"

2. What should happen if user leaves it blank?

Ask again if blank present

Part B – Code with try/except

python

```
def get_int(prompt):
```

```
    while True:
```

```
        user_input = input(prompt)
```

```
        try:
```

```
            value = int(user_input)
```

```
            return value
```

```
        except ValueError:
```

```
            print("Invalid input. Please enter an integer.")
```

If $0 <= \text{value} <= 120$:

else :

*print("please enter a value
b/w 0 & 120")*

```
age = get_int("Enter your age: ")
```

```
print("Your age is:", age)
```

Tasks

- Modify get_int to also enforce a range (e.g., 0 to 120).
- If user enters out-of-range value, show a message and ask again.

*If input is out of range (eg: 150) - ask again to input
only valid integer b/w 0-120 is applicable*

Activity 17 – Mini Project: Text Analyzer

Title: Word Frequency Counter

Problem

Given a short paragraph, count how many times each word appears.

Part A – Plan

Use the problem template (short version):

1. Inputs: paragraph

2. Outputs: word count

3. Steps (bullet points):

- convert to lowercase
- split words
- count frequency

Part B – Implement

python

```
text = input("Enter a sentence/paragraph: ")
```

```
# Convert to lower case
```

```
text = text.lower()
```

```
# Split into words
```

```
words = text.split()
```

```
# Count frequencies using dictionary
```

```
freq = {}
```

```
for w in words:
```

```
    if w in freq:
```

```
        freq[w] += 1
```

```
    else:
```

```
        freq[w] = 1
```

```
# Print results
```

```
for word, count in freq.items():
```

```
    print(word, ":", count)
```

Extensions (optional)

- Remove punctuation before counting.

- Show only words with frequency ≥ 2 .

remove punctuation using string module
print only words with frequency ≥ 2

```
import string
```

```
text = input("Enter text:")
```

```
text = text.lower()
```

```
for p in string.punctuation
```

```
text = text.replace(p, " ")
```

```
words = text.split()
```

Activity 18 – Applying the Full Problem-Solving Workflow

Title: Generic Approach Practice – Loan EMI Calculator

Problem

Compute the Equated Monthly Instalment (EMI) for a loan given:

- Principal (P)
- Annual interest rate (R, in %)
- Tenure in years (N)

Formula (you may give to students):

$$EMI = \frac{P \cdot r \cdot (1+r)^n}{(1+r)^n - 1}$$

where

- $r = R / (12 \times 100)$ (monthly rate)
- $n = N \times 12$ (months)

Use Full Template

1. Problem understanding

- Inputs: P, R, N
- Outputs: EMI
- Constraints: valid numbers

2. Analysis (steps)

- convert rate
- calculate months
- apply formula

3. Design

- Functions: e.g., compute_emi(P, R, N)? ☒ Yes / No
- Data types: float/int? some float

4. Code

python

```
def compute_emi(P, R, N):
```

```
    r = R / (12 * 100)
```

```
    n = N * 12
```

```
    emi = (P * r * (1 + r) ** n) / ((1 + r) ** n - 1)
```

```
    return emi
```

```
P = float(input("Principal: "))
```

```
R = float(input("Annual interest rate (%): "))
```

```
N = int(input("Tenure (years): "))
```

```
emi_value = compute_emi(P, R, N)
```

```
print("Monthly EMI =", emi_value)
```

5. Test

- Test 1: P = 100000, R = 10, N = 2 → EMI = 4614
- Test 2: P = 50000, R = 8, N = 1 → EMI = 4340

Reflection

- Which step (understand/analyse/design/code/test) helped you most?

Design step helps most as it organises logic before coding.